

Tribhuvan University
BSc CSIT 7th Semester

Data Warehousing and Data Mining (CSC410)

Unit 4: Data Cube Technology

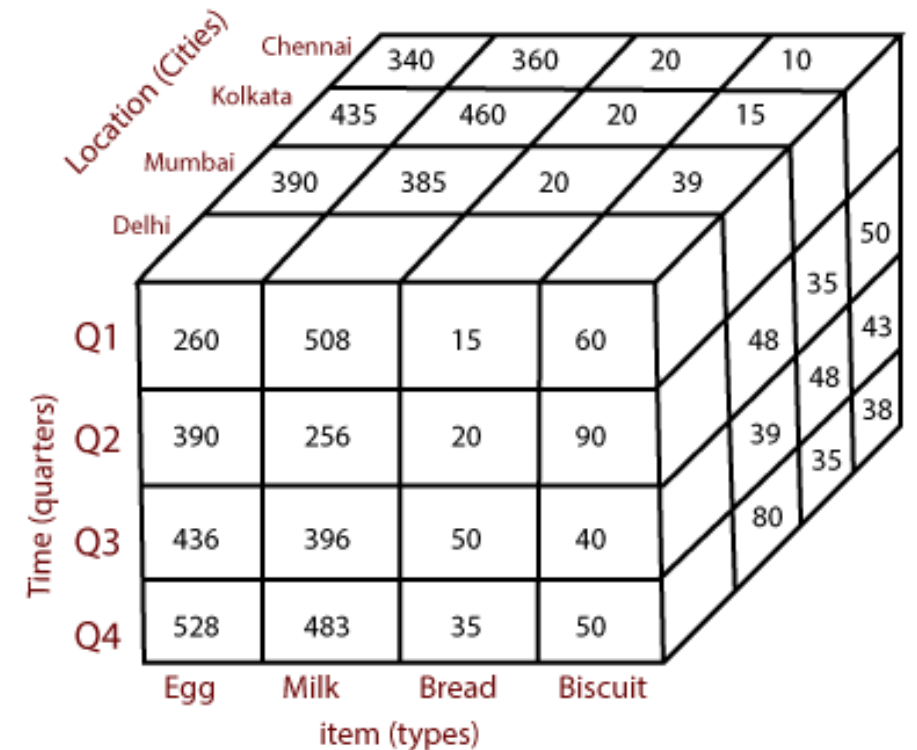
By: Dabbal Singh Mahara
Amrit Campus

Data Cube

- A data cube is a multidimensional data model.
- A data cube allows data to be modeled and viewed in multiple dimensions.
- It is defined by dimensions and facts.
- It extends the idea of a 2-D table to N dimensions
- Used mainly for OLAP (Online Analytical Processing)
- Each dimension represents a perspective of analysis (e.g., Time, Location, Product).
- Each cell in the cube stores a measure (e.g., sales amount, profit).
- Example: Cube = Sales(Product, Location, Time)

Data Cube

- OLAP's multidimensional data structure is known as a cube or a hypercube.
- The cube has one or more dimensions, such as time, geography, and product, and one or more measures, such as sales revenue, units sold, and profit margin.
- The data cube is created by aggregating and summarizing data along multiple dimensions, resulting in a multi-dimensional array or matrix of data.



OLAP Data Cube Structure

- **Aggregation**

- OLAP cubes store data at different levels of aggregation.
- For example, a cube might store sales revenue at the regional level, the state level, and the city level.
- Aggregation is the process of summarizing and consolidating data across different dimensions, such as adding up sales revenue by region or by product category.

- **Hierarchies**

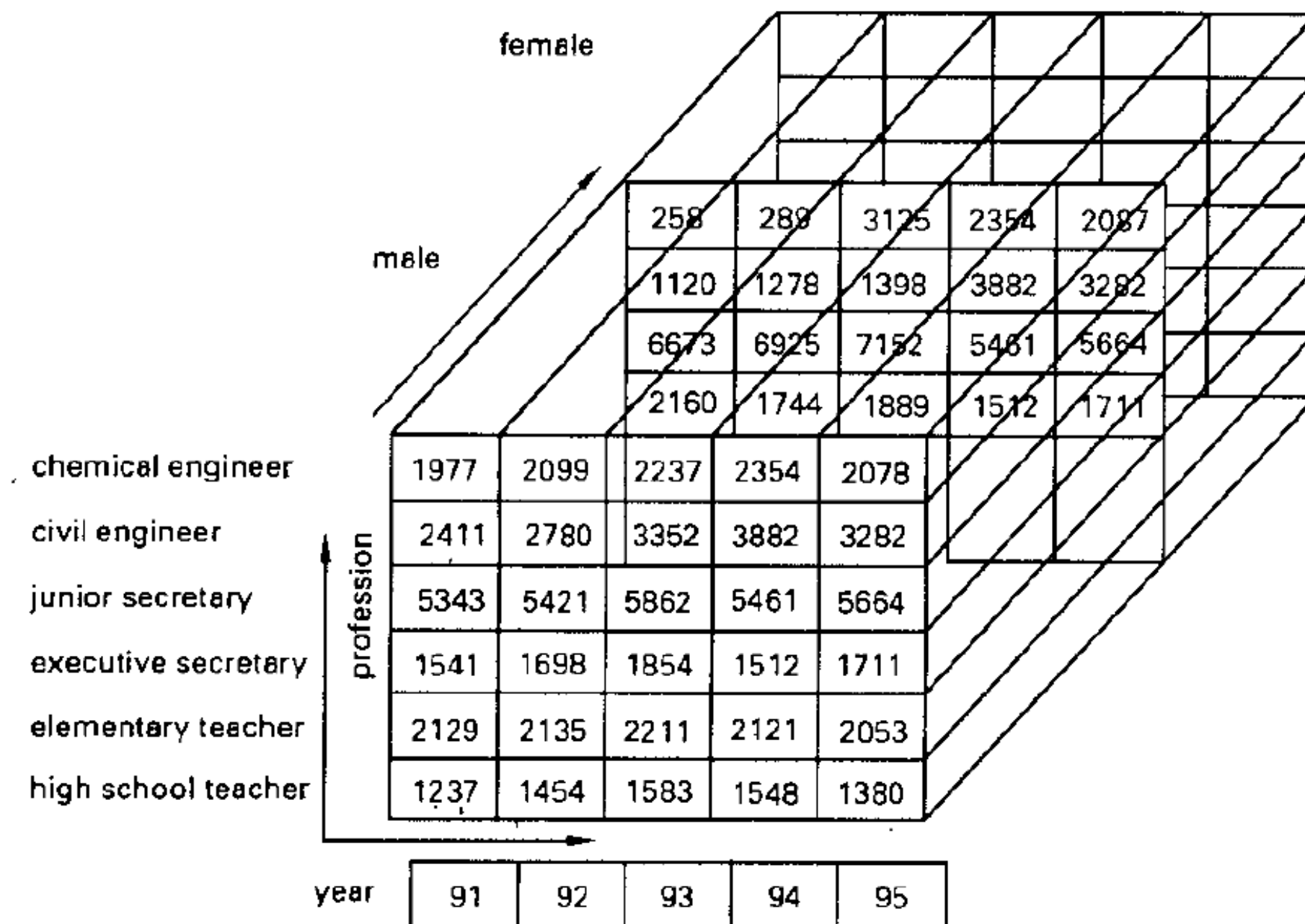
- OLAP data can also include hierarchies, which represent the relationship between different levels of aggregation in a dimension.
- For example, a time hierarchy might include levels for year, quarter, month, and day.

Tabular view of a Data Cube

	Location="Chennai"				Location="Kolkata"				Location="Mumbai"				Location="Delhi"			
	item				item				item				item			
Time	Egg	Milk	Bread	Biscuit	Egg	Milk	Bread	Biscuit	Egg	Milk	Bread	Biscuit	Egg	Milk	Bread	Biscuit
Q1	340	360	20	10	435	460	20	15	390	385	20	39	260	508	15	60
Q2	490	490	16	50	389	385	45	35	463	366	25	48	390	256	20	90
Q3	680	583	46	43	684	490	39	48	568	594	36	39	436	396	50	40
Q4	535	694	39	38	335	365	83	35	338	484	48	80	528	483	35	50

Multidimensional Data View

			Professional Class					
			Engineer		Secretary		Teaching	
			PROFESSION		PROFESSION		PROFESSION	
			Chemical Engineers	Civil Engineers	Junior Secretary	Executive Secretary	Elementary Teachers	High School Teachers
SEX	M A L E	91	1977	2411	5343	1541	2129	1237
		92	2099	2780	5421	1698	2135	1457
		93	2237	3352	5862	1854	2211	1583
		94	2354	3882	5461	1512	2121	1548
		95	2078	3282	5664	1711	2053	1380
	F E M A L E	91	258	1120	6673	1623	2160	2751
		92	289	1276	6925	1744	2175	2993
		93	312	1398	7152	1889	2189	3125
		94	581	1216	6543	1534	2857	2387
		95	329	1321	6129	1567	2453	3287



<i>location</i> = “Vancouver”				
<i>time</i> (quarter)	<i>item</i> (type)			
	<i>home</i>			
	<i>entertainment</i>	<i>computer</i>	<i>phone</i>	<i>security</i>
Q1	605	825	14	400
Q2	680	952	31	512
Q3	812	1023	30	501
Q4	927	1038	38	580

Table: A 2-D view of sales data according to the dimensions *time* and *item*, where the sales are from branches located in the city of Vancouver. The measure displayed is *dollars sold* (in thousands).

<i>location</i> = “Chicago”					<i>location</i> = “New York”				<i>location</i> = “Toronto”				<i>location</i> = “Vancouver”			
<i>item</i>					<i>item</i>				<i>item</i>				<i>item</i>			
<i>home</i>					<i>home</i>				<i>home</i>				<i>home</i>			
<i>time</i>	<i>ent.</i>	<i>comp.</i>	<i>phone</i>	<i>sec.</i>	<i>ent.</i>	<i>comp.</i>	<i>phone</i>	<i>sec.</i>	<i>ent.</i>	<i>comp.</i>	<i>phone</i>	<i>sec.</i>	<i>ent.</i>	<i>comp.</i>	<i>phone</i>	<i>sec.</i>
Q1	854	882	89	623	1087	968	38	872	818	746	43	591	605	825	14	400
Q2	943	890	64	698	1130	1024	41	925	894	769	52	682	680	952	31	512
Q3	1032	924	59	789	1034	1048	45	1002	940	795	58	728	812	1023	30	501
Q4	1129	992	63	870	1142	1091	54	984	978	864	59	784	927	1038	38	580

Table: A 3-D view of sales data according to the dimensions *time*, *item*, and *location*.
The measure displayed is *dollars sold* (in thousands).

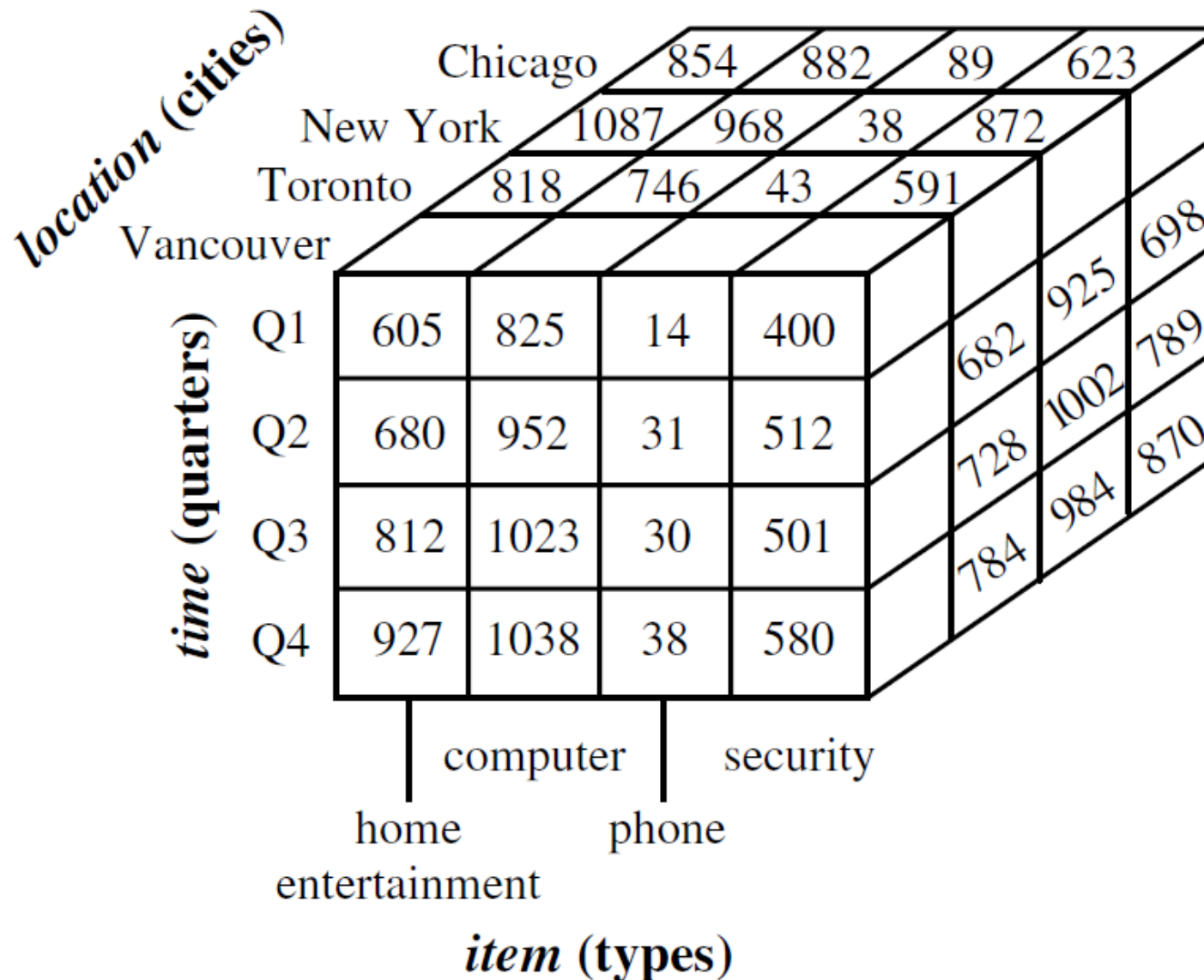


Figure: A 3-D data cube representation of the data in the table above, according to the dimensions *time*, *item*, and *location*. The measure displayed is *dollars sold* (in thousands).

Question?

- Suppose that we would now like to view our sales data with an additional fourth dimension, such as *supplier*.
- What should we do?
- Any Solution?

Solution!!

Viewing things in 4-D becomes tricky. However, we can think of a 4-D cube as being a series of 3-D cubes as shown below:

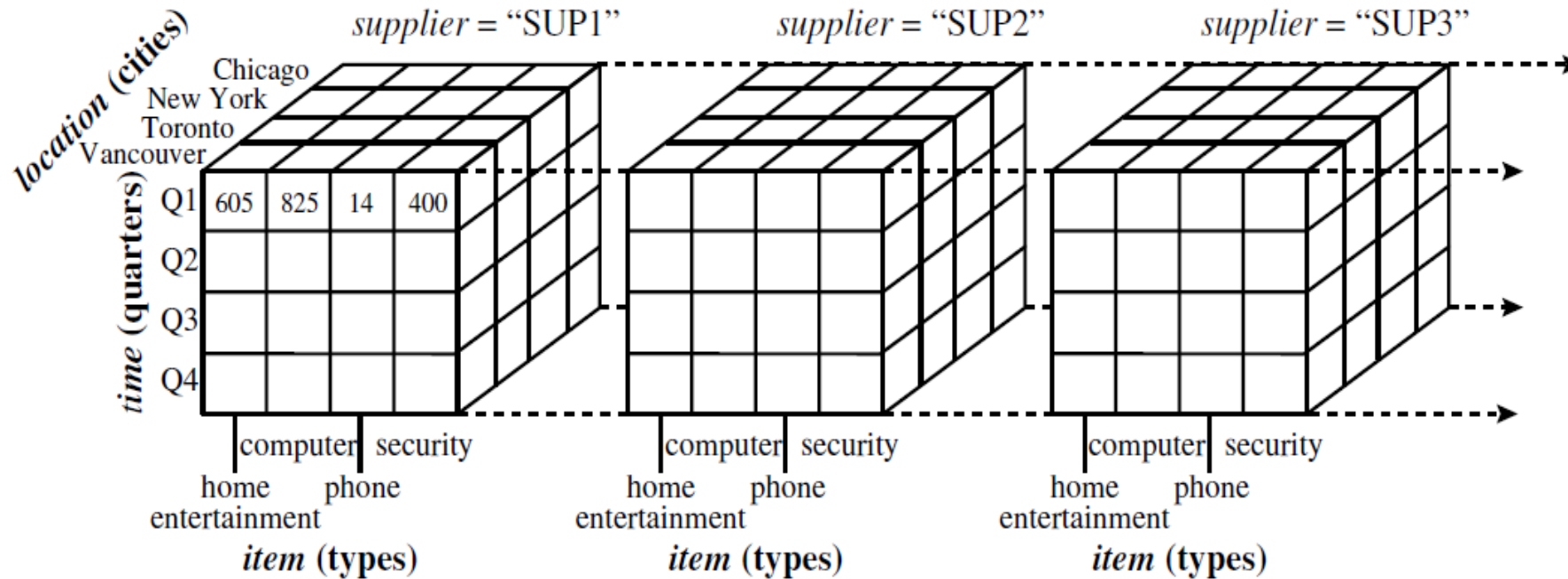


Figure: A 4-D data cube representation of sales data, according to the dimensions *time*, *item*, *location*, and *supplier*. The measure displayed is *dollars sold* (in thousands). For improved readability, only some of the cube values are shown.

Cuboids

- In data warehousing literature, a cuboid is a specific aggregation level (or view) of a data cube.
- The most detailed part of the cube is called a **base cuboid**.
- The top-most 0-D cuboid, which holds the highest-level of summarization, is called the **apex cuboid**.
- Different cuboids show data at different levels of detail.

		year				
		1999	2000	2001	2002	ALL
product	chairs	25	37	89	21	172
	tables	10	30	0	45	85
	desks	56	84	9	35	184
	shelves	19	20	0	71	110
	boards	5	16	11	15	47
	ALL	115	187	109	187	598

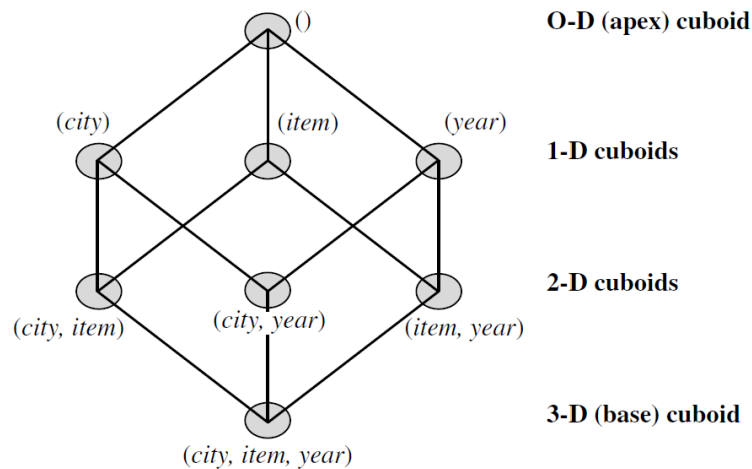
base cuboid

Data cube

apex cuboid

Lattice of Cuboids

- The lattice of cuboids is the collection of all possible cuboids formed by different combinations of dimensions in a data cube.
- Cuboids in a lattice represent all possible aggregated views of a data cube arranged according to their levels of aggregation.
- The top cuboid contains the most summarized data.
- The bottom cuboid (base cuboid) contains the most detailed data.



- moving upward (roll-up) → more aggregation
- moving downward (drill-down) → more detail

Lattice of cuboids, making up a 3-D data cube. Each cuboid represents a different group-by. The base cuboid contains *city*, *item*, and *year* dimensions.

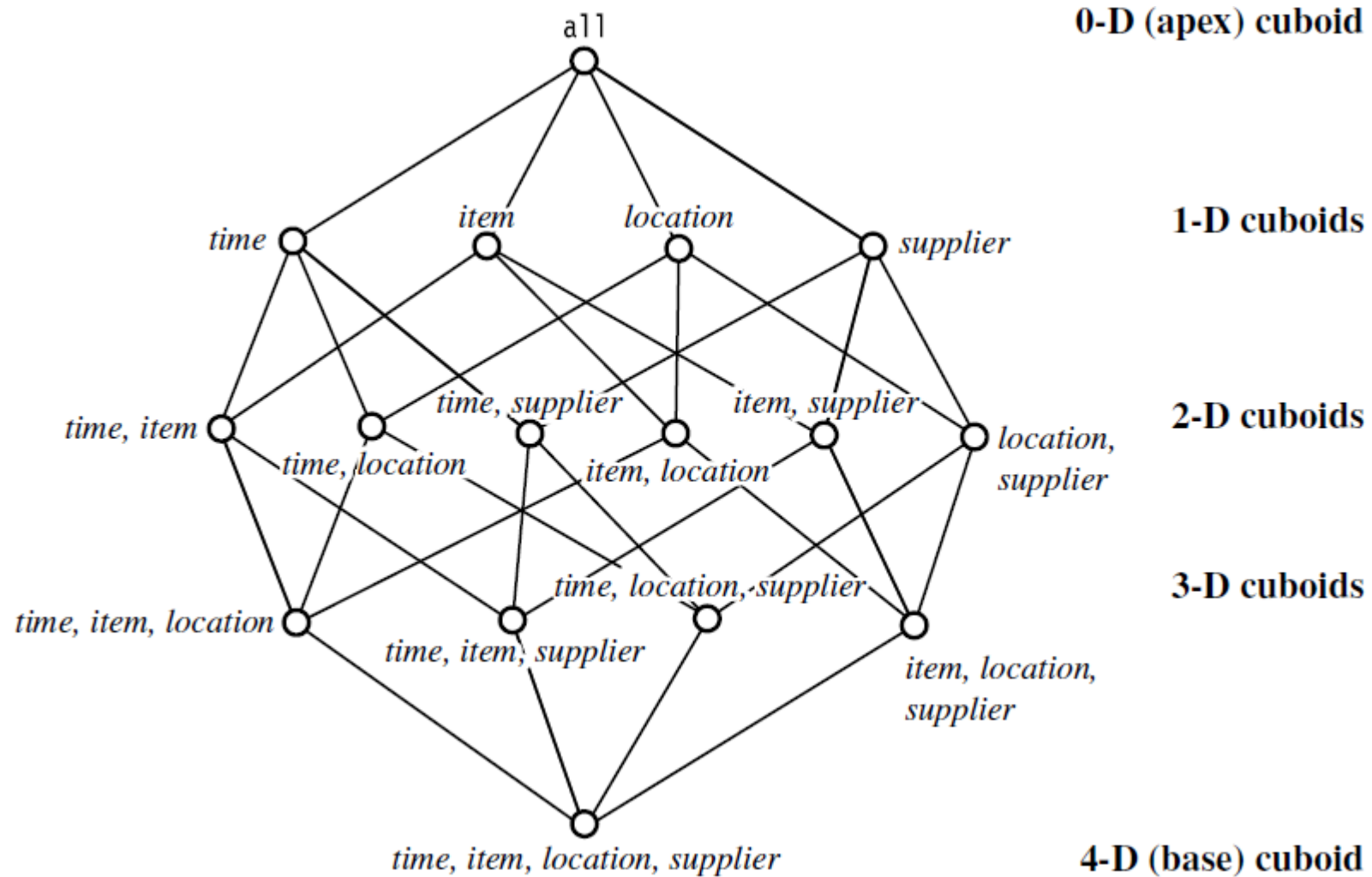


Figure: Lattice of cuboids, making up a 4-D data cube for the dimensions *time*, *item*, *location*, and *supplier*. Each cuboid represents a different degree of summarization.

Efficient Computation of Data Cubes

- At the core of multidimensional data analysis is the efficient computation of aggregations across many sets of dimensions.
- In SQL terms, these aggregations are referred to as group-by's.
- Each group-by can be represented by a cuboid, where the set of group-by's forms a lattice of cuboids defining a data cube.
- We explore issues relating to the efficient computation of data cubes.

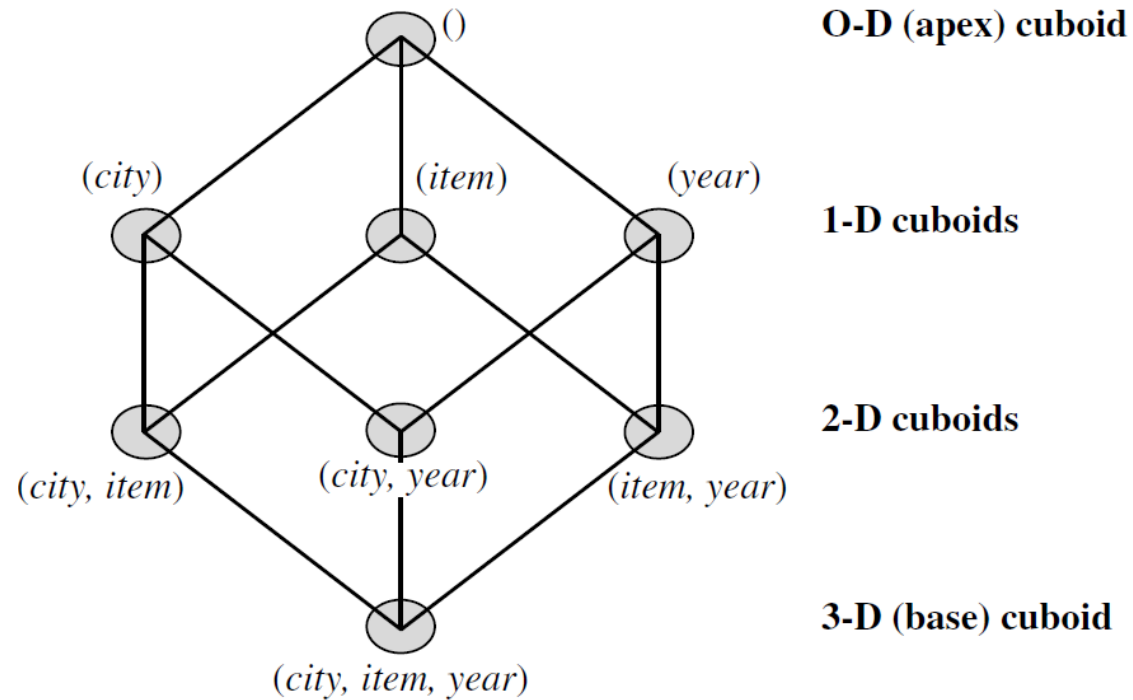
The compute cube Operator

- One approach to cube computation extends SQL so as to include a compute cube operator.
- The compute cube operator computes aggregates over all subsets of the dimensions specified in the operation. This can require excessive storage space, especially for large numbers of dimensions.
- **Example :** A data cube is a lattice of cuboids. Suppose that we want to create a data cube for *AllElectronics* sales that contains the following: *city*, *item*, *year*, and *sales in dollars*.
- We want to be able to analyze the data, with queries such as the following:
 - “Compute the sum of sales, grouping by city and item.”
 - “Compute the sum of sales, grouping by city.”
 - “Compute the sum of sales, grouping by item.”

Example

- What is the total number of cuboids, or group-by's, that can be computed for this data cube?
- Taking the three attributes, *city*, *item*, and *year*, as the dimensions for the data cube, and *sales in dollars* as the measure, the total number of cuboids, or groupby's, that can be computed for this data cube is $2^3 = 8$.
- The possible group-by's are the following: $\{(city, item, year), (city, item), (city, year), (item, year), (city), (item), (year), ()\}$, where () means that the group-by is empty (i.e., the dimensions are not grouped).

Example



Lattice of cuboids, making up a 3-D data cube. Each cuboid represents a different group-by. The base cuboid contains *city*, *item*, and *year* dimensions.

Example

- The **base cuboid** contains all three dimensions, *city*, *item*, and *year*. It can return the total sales for any combination of the three dimensions.
- The **apex cuboid**, or 0-D cuboid, refers to the case where the group-by is empty. It contains the total sum of all sales.
- The base cuboid is the least generalized (most specific) of the cuboids.
- The apex cuboid is the most generalized (least specific) of the cuboids, and is often denoted as all.
- If we start at the apex cuboid and explore downward in the lattice, this is equivalent to drilling down within the data cube.
- If we start at the base cuboid and explore upward, this is akin to rolling up.

How many cuboids are there in an n-dimensional data cube?

- If there were no hierarchies associated with each dimension, then the total number of cuboids for an n -dimensional data cube, as we have seen, is 2^n .
- However, in practice, many dimensions do have hierarchies.
- For example, *time* is usually explored not at only one conceptual level (e.g., *year*), but rather at multiple conceptual levels such as in the hierarchy “*day < month < quarter < year*.”
- For an n -dimensional data cube, the total number of cuboids that can be generated (including the cuboids generated by climbing up the hierarchies along each dimension) is

$$\text{Total number of cuboids} = \prod_{i=1}^n (L_i + 1)$$

where L_i is the number of levels associated with dimension i . One is added to L_i in to include the *virtual* top level, all.

- **Example :** If the cube has 10 dimensions and each dimension has 4 levels, what will be the number of cuboids generated?

Solution: Here $n=10$ $L_i=4$ for $i=1,2,\dots,10$

Thus, Total number of cuboids = $5 \times 5 \times 5 \times 5 \times 5 \times 5 \times 5 \times 5 \times 5 \times 5 = 5^{10} \approx 9.8 \times 10^6$

- From above example, it is unrealistic to pre-compute and materialize all of the cuboids that can possibly be generated for a data cube (or from a base cuboid).
- If there are many cuboids, and these cuboids are large in size, a more reasonable option is partial materialization, that is, to materialize only some of the possible cuboids that can be generated.

Partial Materialization: Selected Computation of Cuboids

There are three choices for data cube materialization given a base cuboid:

- a. **No materialization:** Do not precompute any of the “nonbase” cuboids. This leads to computing expensive multidimensional aggregates on-the-fly, which can be extremely slow.
- b. **Full materialization** – It can Pre-compute all of the cuboids. The resulting lattice of computed cuboids is defined as the full cube. This choice typically requires huge amounts of memory space to store all of the precomputed cuboids.
- c. **Partial materialization** – It can selectively calculate a proper subset of the whole set of possible cuboids. Alternatively, it can calculate a subset of the cube, which includes only those cells that satisfy some user-specified criterion, including where the tuple count of each cell is following some threshold.

Cube materialization (Full cube, Iceberg cube, Closed cube, Shell cube)

- **Full cube:** Full materialization, In order to ensure fast OLAP, it is sometimes desirable to precompute the full cube (i.e., all the cells of all of the cuboids for a given data cube). This, however, is exponential to the number of dimensions. That is, a data cube of n dimensions contains 2^n cuboids. There are even more cuboids if we consider concept hierarchies for each dimension.
- In addition, the size of each cuboid depends on the cardinality of its dimensions. Thus, precomputation of the full cube can require huge and often excessive amounts of memory.
- **Iceberg cube:** Partially materialized, it contains only those cuboid cells whose measures satisfy the aggregate or iceberg condition.
- The aggregate condition could be, minimum support or a lower bound on average, min or max.
- It is called an iceberg cube because it contains only some of the cells of the full cube like the tip of an iceberg.

- The purpose of the Iceberg-cube is to identify and compute only those values that will most likely be required for decision support queries. The aggregate condition specifies which cube values are more meaningful and should therefore be stored. This is one solution to the problem of computing versus storing data cubes.
- An iceberg cube can be specified with an SQL query,

Example 1:

```
compute cube sales iceberg as  
select month, city, customer group, count(*)  
from salesInfo  
  
cube by month, city, customer_group  
having count(*) >= min_sup
```

Example 2

If sales data is analyzed and the condition is:

Store only cuboid cells where **total sales $\geq 1,00,000$**

Then:

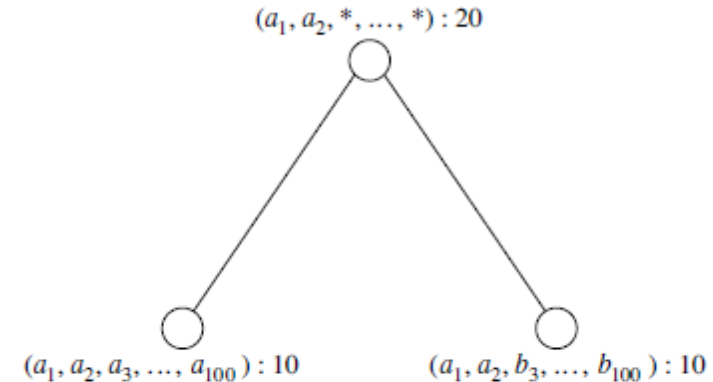
High-value sales data → stored in iceberg cube

Low-value sales data → discarded

- The compute cube statement specifies the precomputation of the iceberg cube, *sales iceberg*, with the dimensions *month*, *city*, and *customer group*, and the aggregate measure count(). The input tuples are in the *salesInfo* relation. The cube by clause specifies that aggregates (group-by's) are to be formed for each of the possible subsets of the given dimensions. If we were computing the full cube, each group-by would correspond to a cuboid in the data cube lattice. The constraint specified in the having clause is known as the **iceberg condition**. Here, the iceberg measure is count().

Closed cube:

- Closed Cube is an optimized, compressed form of a data cube where only closed cells are materialized to save storage while retaining full informational value.
- In a closed cube, only those cuboids are stored whose aggregate values cannot be obtained from any higher-level cuboid with the same value.
- A cell c is considered "closed" if no descendant (specialized) cell d exists that has the same measure value as c .
- if aggregate value is unique at that level and cannot be obtained from a broader category, we store it; otherwise, we ignore it.
- This avoids storing redundant aggregates.



! Three closed cells forming the lattice of a closed cube.

- **Shell cube:** Another strategy for partial materialization is to pre-compute only portions and fragments of the cube shell (cuboids involving small no. of dimensions such as 3 to 5), based on the cuboids of interest.
- This data cube in OLAP that stores only the minimal set of cuboids needed to efficiently compute any other aggregate in the full cube.
- For example, shell fragment cube ABC contains 7 cuboids: A, B, C, AB, AC, BC, and ABC).

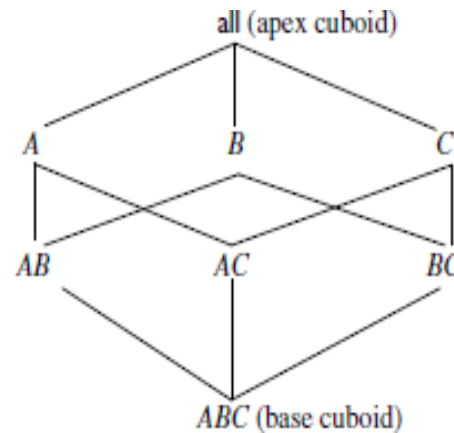


Figure: Lattice of cuboids making up a 3-D data cube with the dimensions A , B , and C for some aggregate measure, M

General strategies for cube computation

- General strategies for data cube computation are designed to enhance efficiency and scalability.
- These strategies aim to reduce the time required to construct data cubes.
- They also help in minimizing the storage space needed for multidimensional analysis.
- The selected strategy depends on the type of OLAP system being used.
- In Relational OLAP (ROLAP), data cubes are generated using relational database tables.
- In Multidimensional OLAP (MOLAP), data cubes are built using multidimensional array-based structures.

ROLAP vs MOLAP

SN	Aspect	ROLAP (Relational OLAP)	MOLAP (Multidimensional OLAP)
1	Storage model	Uses relational tables in a relational database	Uses multidimensional array-based storage
2	Data storage	Data remains in fact and dimension tables	Data is stored in pre-computed data cubes
3	Query performance	Slower compared to MOLAP due to SQL joins	Faster because data is pre-aggregated
4	Scalability	Highly scalable for very large datasets	Limited scalability for very large datasets
5	Storage space	Requires less storage as data is not fully pre-computed	Requires more storage due to cube materialization
6	Computation	Aggregations are computed at query time	Aggregations are computed in advance
7	Complexity	Simpler to implement and manage	More complex to design and maintain
8	Flexibility	More flexible for schema changes	Less flexible once cubes are built
9	Typical use	Suitable for large, detailed datasets	Suitable for fast analytical queries

General strategies for cube computation

1. Sorting, Hashing, and Grouping in Data Cube Computation

- In data cube computation, aggregation (such as SUM, COUNT, AVG) is performed on tuples or cells that have the same combination of dimension values.
To make this aggregation efficient, the data is first reordered or clustered using sorting, hashing, or grouping operations on dimension attributes.
- For example, when computing total sales by branch, day, and item, the data can be sorted by branch, then by day, and then grouped by item.
- This arrangement places all related tuples together, allowing aggregate functions such as SUM to be applied quickly.
- Instead of sorting, hashing can be used to partition tuples based on the same set of dimension values, enabling aggregation within each partition.

General strategies for cube computation

2. Simultaneous Aggregation and Caching:

- It is more efficient to compute higher-level (more generalized) aggregates from previously computed lower-level aggregates rather than from the base fact table.
- Unit To compute sales by *branch*, for example, we can use the intermediate results derived from the computation of a lower-level cuboid such as sales by *branch* and *day*.
- By simultaneously aggregating multiple cuboids from cached intermediate results, a system can significantly reduce expensive disk I/O operations through amortized scans

General strategies for cube computation

3. Aggregation from the smallest child when there exist multiple child cuboids:

- When there exist multiple child cuboids, it is usually more efficient to compute the desired parent (i.e., more generalized) cuboid from the smallest, previously computed child cuboid.
- To compute a sales cuboid, C_{branch} , when there exist two previously computed cuboids, $C_{\{branch,year\}}$ and $C_{\{branch,item\}}$, for example, it is obviously more efficient to compute C_{branch} from the former than from the latter if there are many more distinct items than distinct years.

4. Apriori Pruning for Iceberg Cubes:

- This technique is essential when computing iceberg cubes, which only materialize cells satisfying a minimum support threshold.
- The Apriori property states that if a cell does not satisfy the minimum support, none of its descendants (more specialized cells) will satisfy it either; this allows the algorithm to prune large portions of the search space.

Attribute oriented induction for data characterization

- Attribute-Oriented Induction (AOI) is a data mining technique used to generalize detailed data into higher-level concepts using concept hierarchies to produce summarized information or rules.
- Example:
City $\rightarrow$ Province $\rightarrow$ Country
Pokhara $\rightarrow$ Gandaki $\rightarrow$ Nepal
- AOI replaces detailed values (City) with generalized values (Country) to simplify data analysis.
- **Purpose:** Characterize a target class (e.g., graduate students, high-value customers).
- **Method:** Replace specific attribute values with higher-level concepts.
- **Tools:** Uses concept hierarchies for generalization.
- **Output:** Generalized tuples, aggregated counts, and characteristic rules.

Example

Original Data:

Age	Major	City
23	CS	Kathmandu
25	IT	Lalitpur

After AOI:

Age Group	Field	Region
Youth	Computing	Bagmati

Characterization Rule:

“Graduate students are mostly **youth**, in **computing fields**, from the **Bagmati region**.”

Attribute oriented induction for data characterization

- The Attribute-Oriented Induction (AOI) approach to data generalization and summarization-based characterization was first proposed in 1989, a few years before the introduction of the data cube approach.
- The data cube approach can be considered as a data warehouse-based, pre-computational-oriented, materialized approach. It performs offline aggregation before an OLAP or data mining query is submitted for processing.
- On the other hand, the attribute-oriented induction approach, at least in its initial proposal, is a relational database query-oriented, generalized-based, online data analysis technique.
- However, there is no inherent barrier distinguishing the two approaches based on online aggregation versus offline pre-computation.

Basic Principles of Attribute Oriented Induction

1. Data Focusing: Selection of Task-Relevant Data

- Data focusing means selecting only the data that is relevant to the data mining task. The selected dataset is called the **initial relation**.
- In real databases, many attributes exist, but not all are useful. AOI first selects only relevant attributes and records needed for analysis. This reduces unnecessary data processing.

2. Attribute Removal

- An attribute is removed if:
 - It has many distinct values, and
 - There is no concept hierarchy for generalization, OR
 - Its information can be represented using other attributes.
- Some attributes cannot be generalized meaningfully.
- Unique identifiers like Student_ID, Name are usually removed.

Basic Principles of Attribute Oriented Induction

3. Attribute Generalization

- Attribute generalization is the process of replacing detailed, low-level attribute values with higher-level concepts using a concept hierarchy to reduce data complexity and summarize information.
- **Example: City → Province → Country**
- This hierarchy allows generalization from specific to more general values.

Original Data (City level – detailed)

Student	City
S1	Pokhara
S2	Kathmandu
S3	Lalitpur
S4	Biratnagar

Step 1: Generalize City → Province

Using concept hierarchy:

Student	Province
S1	Gandaki
S2	Bagmati
S3	Bagmati
S4	Koshi

Now the number of distinct values is reduced.

Step 2: Further Generalize Province → Country

Student	Country
S1	Nepal
S2	Nepal
S3	Nepal
S4	Nepal

Now data is fully generalized and easier to analyze.

- **City has many distinct values,** making analysis complex.

Basic Principles of Attribute Oriented Induction

4. Attribute Threshold Control

- Attribute threshold control is the process of limiting the maximum number of distinct values allowed for an attribute. If the number of distinct values exceeds the specified threshold, the attribute is generalized to a higher-level concept using a concept hierarchy.
- It's purpose is to reduce data complexity, simplify data representation and to control the level of generalization

Example: Concept hierarchy: City $\rightarrow$ Province $\rightarrow$ Country
Threshold = 2

Before Generalization

City
Pokhara
Kathmandu
Lalitpur
Biratnagar

Distinct values = 4 (> 2)
 $\rightarrow$ Exceeds threshold



Step 1: Generalize to Province

Province
Gandaki
Bagmati
Bagmati
Koshi

Distinct values = 3 (> 2)
 $\rightarrow$ Still exceeds threshold



Step 2: Generalize to Country

Country
Nepal
Nepal
Nepal
Nepal

Distinct values = 1 (≤ 2)
 $\rightarrow$ Within threshold

Algorithm for Attribute Oriented Induction

- **InitialRel:** It is nothing but query processing of task-relevant data and deriving the initial relation.
- **PreGen:** It is based on the analysis of the number of distinct values in each attribute and to determine the generalization plan for each attribute: removal? or how high to generalize?
- **PrimeGen:** It is based on the PreGen plan and performing the generalization to the right level to derive a “prime generalized relation” and also accumulating the counts.
- **Presentation:** User interaction: (1) adjust levels by drilling, (2) pivoting, (3) mapping into rules, cross tabs, visualization presentations.

Example:

Let's say there is a University database that is to be characterized, for that its corresponding DMQL will be

use University_DB

mine characteristics as “Science_Students”

in relevance to name, gender, major, birth_place, birth_date, residence, phone_no, GPA

from student

Its corresponding SQL statement can be:

Select name, gender, major, birth_place, birth_date, residence, phone_no, GPA

from student

where status in {“Msc”, “MBA”, “Ph.D.” }

Now for this database let's create a characterized view:

InitialRel:

- From this table, we are querying task-relevant data.
- From this table, we also removed a few attributes like name and phoneno, because they make no sense in concluding insights.

PreGen :

- Now, we have generalized these results by removing a few attributes and retaining important attributes.
- And also we have generalized a few attributes by naming them "Country" rather than "Birth_Place", "Age Range" rather than "Birth_data", "City" rather than "Residence" and so on as per the table given below.

Name	Gender	Major	Birth-Place	Birth_date	Residence	Phone #	GPA
Jim Woodman	M	CS	Vancouver,BC, Canada	8-12-76	3511 Main St., Richmond	687-4598	3.67
Scott Lachance	M	CS	Montreal, Que, Canada	28-7-75	345 1st Ave., Richmond	253-9106	3.70
Laura Lee	F	Physics	Seattle, WA, USA	25-8-70	125 Austin Ave., Burnaby	420-5232	3.83
...	...	...	...	...	...	...	...
Removed	Retained	Sci,Eng, Bus	Country	Age range	City	Removed	Excl, VG,...

PrimeGen:

- Based on the PreGen plan we've performed generalization to the right level to derive a “prime generalized relation” and also we've accumulated the counts.

Gender	Major	Birth_region	Age_range	Residence	GPA	Count
M	Science	Canada	20-25	Richmond	Very-good	16
F	Science	Foreign	25-30	Burnaby	Excellent	22
...	...	...	...	...	...	...

Final Results :

- Now we've analyzed and concluded our final generalized results as shown below.

Gender	Birth_Region		
	Canada	Foreign	Total
M	16	14	30
F	10	22	32
Total	26	36	62

Presentation Of Results

Generalized relation: Relations where some or all attributes are generalized, with counts or other aggregation values accumulated.

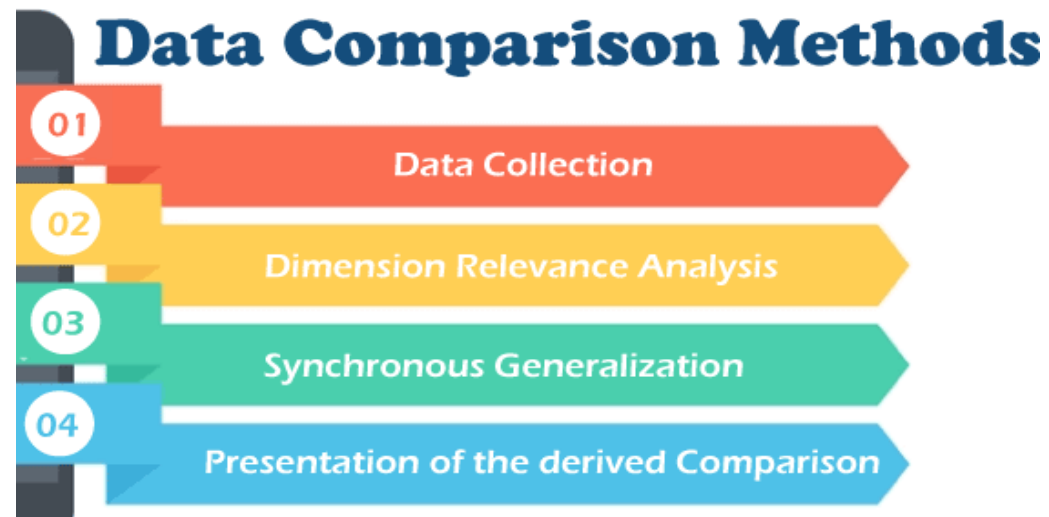
Cross-tabulation: Mapping results into cross-tabulation form (similar to contingency tables).

Visualization techniques: Pie charts, bar charts, curves, cubes, and other visual forms.

Quantitative characteristic rules: Mapping generalized results in characteristic rules with quantitative information associated with it.

Mining class comparison: discriminating between different classes

- In many applications, users may not be interested in having a single class described or characterized, but rather would prefer to mine a description that compares or distinguishes one class from other comparable classes.
- Class discrimination or comparison mines descriptions that distinguish a target class from its contrasting classes. Note that the target and contrasting classes must be comparable in the sense that they share similar dimensions and attributes.
- For example, the three classes, person, address, and item are not comparable. The procedures for class comparison performed as follows:



1. **Data Collection:** The set of associated data from the databases and data warehouses is collected by query processing and is partitioned into the target class and contrasting class(es).
2. **Dimension Relevance Analysis:** When many dimensions are to be processed and is required that analytical comparison should be performed, then dimension relevance analysis should be performed on these classes, and only the highly relevant dimensions are included in the further analysis.
3. **Synchronous Generalization:** The process of generalization is performed upon the class to the level controlled by the user or expert specified dimension threshold, which results in a prime target class relation/cuboid. The concepts in the contrasting class or classes are generalized to the same level as those in the prime target class relation/cuboid, forming the prime contrasting class relation/cuboid. Target.
4. **Presentation of the derived comparison:** The resulting class comparison description can be visualized in the form of tables, charts, and rules. This presentation usually includes a “contrasting” measure (such as count%) that reflects the comparison between the target and contrasting classes.

Example:

Task - Compare graduate and undergraduate students using the discriminant rule. for this, the DMQL query would be.

use University_Database

mine comparison as “graduate_students vs_undergraduate_students”

in relevance to name, gender, program, birth_place, birth_date, residence, phone_no, GPA

for “graduate_students”

where status in “graduate”

versus “undergraduate_students”

where status in “undergraduate”

analyze count%

from student

Now from this, we can formulate that

- **attributes** = name, gender, program, birth_place, birth_date, residence, phone_no, and GPA.
 - **Gen(ai)** = concept hierarchies on attributes ai.
 - **Ui** = attribute analytical thresholds for attributes ai.
 - **Ti** = attribute generalization thresholds for attributes ai.
 - **R** = attribute relevance threshold.
1. Data collection -Understanding **Target** and **Contrasting** classes.
 2. Attribute relevance analysis - It is used to **remove attributes** name, gender, program, phone_no.
 3. Synchronous generalization - It is controlled by user-specified dimension thresholds, **a prime target, and contrasting class(es)** relations/cuboids.

Initial target class working relation (graduate student)

Name	Gender	Major	Birth-Place	Birth_date	Residence	Phone #	GPA
Jim Woodman	M	CS	Vancouver,BC, Canada	8-12-76	3511 Main St., Richmond	687-4598	3.67
Scott Lachance	M	CS	Montreal, Que, Canada	28-7-75	345 1st Ave., Richmond	253-9106	3.70
Laura Lee	F	Physics	Seattle, WA, USA	25-8-70	125 Austin Ave., Burnaby	420-5232	3.83
...	...	...	...	...	...	...	...

Initial contrasting class working relation (graduate student)

Name	Gender	Major	Birth-Place	Birth_date	Residence	Phone #	GPA
Bob Schumann	M	Chem	Calagary, Alt, Canada	10-1-78	2642 Halifax St, Burnaby	294-4291	2.96
Ammy. Eau	F	Bio	Golden, BC, Canada	30-3-76	463 Sunset Cres, Vancouver	681-5417	3.52
...	...	...	...	...	...	...	...

4. Drill down, roll up and other OLAP operations on target and contrasting classes to adjust levels of abstractions of resulting description.

Prime generalized relation for the target class: Graduate students

Major	Age_range	Gpa	Count%
Science	20-25	Good	5.53%
Science	26-30	Good	2.32%
Science	Over_30	Very_good	5.86%
...	...	...	...
Business	Over_30	Excellent	4.68%

Prime generalized relation for the contrasting class: Undergraduate students

Major	Age_range	Gpa	Count%
Science	15-20	Fair	5.53%
Science	15-20	Good	4.53%
...	...	...	...
Science	26-30	Good	5.02%
...	...	...	...
Business	Over_30	Excellent	0.68%

5. The presentation- Data is presented as generalized relations, crosstabs, bar charts, pie charts, or rules, contrasting measures to reflect a comparison between target and contrasting classes.

e.g. count%

Thank You !